

算法竞赛 C++ 模板

从竞赛到禁赛

Asanagi_Sagiri

这份手册汇总了算法竞赛中常用的 C++ 模板，覆盖数论与组合计数、数据结构、图论、128 位整数、字符串、AC 自动机、动态规划与调试工具。示例默认使用 GNU C++17；代码以可复制、边界明确、注释清晰和便于赛时修改为目标。

目录

- [使用约定](#)
- [1. 数论](#)
- [2. 数据结构](#)
- [3. 图论](#)
- [4. 128 位整数](#)
- [5. 精确整数函数](#)
- [6. 字符串](#)
- [7. 实用工具](#)
- [8. 动态规划](#)
- [9. 基础算法补充](#)
- [复杂度速查](#)
- [在线评测实测](#)

使用约定

- 头文件统一使用 `#include <bits/stdc++.h>`，并保留 `using namespace std;`。
- 索引、点编号和容器长度使用 `int`；答案、距离和乘积优先使用 `ll`。
- 不使用 `#define int long long`、`#define endl`、`pb` 等宏，避免类型和接口被隐式改变。
- 图、树、并查集、树状数组和线段树默认 **1-based**；字符串和普通数组算法默认 **0-based**。
- 类型名使用 `PascalCase`，多词函数与变量使用 `snake_case`，缩进为 4 个空格。
- 代码注释按“状态含义、维护方式、边界条件、易错点”的顺序编写；赛时确认理解后可按需删除。
- 同一章节的后续片段可能依赖前文类型，例如 Dijkstra、SPFA 和 Prim 依赖 `Edge` 与 `Graph`，模逆元依赖 `exgcd`。

比赛代码骨架

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;           // 常规 64 位有符号整数。
using i128 = __int128_t;        // 需要避免 64 位乘法溢出时使用。
```

```

using u128 = __uint128_t;
using pii = pair<int, int>; // 常用的“两个 int”组合。

void Asanagi()
{
    // 当前测试用例的主逻辑。
}

int main()
{
    // 关闭 C 与 C++ 标准流同步，解绑 cin/cout，可显著加快输入输出。
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // 默认只有一组数据：题目明确给出 T 时再取消下一行注释。
    int t = 1;
    // cin >> t;
    while (t--)
    {
        Asanagi();
    }
    return 0;
}

```

1. 数论

1.1 64 位素数测试与整数分解

确定性 Miller-Rabin 可在 `uint64_t` 范围内判断素数；Pollard-Rho 用于分解没有小因子的合数。前置条件为 `n >= 1`，分解结果按升序返回。

- Miller-Rabin：对 64 位整数使用固定底数，单次判定约为 $O(\log n)$ 次模乘。
- Pollard-Rho：期望复杂度约为 $O(n^{1/4})$ ，最坏复杂度不作保证。
- 模乘使用 `__uint128_t`，避免 64 位乘法溢出。

```

using u64 = uint64_t;
using u128 = __uint128_t;

u64 mul_mod(u64 a, u64 b, u64 mod)
{
    // 先提升到 128 位再相乘，避免 a * b 在 64 位中溢出。
    return static_cast<u64>(static_cast<u128>(a) * b % mod);
}

u64 pow_mod(u64 a, u64 b, u64 mod)
{
    // res 保存已经选中的二进制位所贡献的乘积。
    u64 res = 1 % mod;
    while (b > 0)
    {
        // 当前最低位为 1，说明答案需要乘上当前的 a。
    }
}

```

```

        if (b & 1)
        {
            res = mul_mod(res, a, mod);
        }
        // a 依次表示原底数的 1、2、4、8.....次幂。
        a = mul_mod(a, a, mod);
        b >>= 1;
    }
    return res;
}

bool is_prime(u64 n)
{
    // 0、1 以及负数语义下的值都不是素数。
    if (n < 2)
    {
        return false;
    }

    // 先处理小质数，既能加速，也能避免后面出现特殊边界。
    for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL, 31ULL,
37ULL})
    {
        if (n % p == 0)
        {
            return n == p;
        }
    }

    // 把  $n - 1$  写成  $d * 2^s$ ，其中  $d$  为奇数。
    u64 d = n - 1;
    int s = 0;
    while ((d & 1) == 0)
    {
        d >>= 1;
        ++s;
    }

    // 这组底数足以确定性判断完整 uint64_t 范围。
    constexpr u64 bases[] = {
        2ULL, 325ULL, 9375ULL, 28178ULL,
        450775ULL, 9780504ULL, 1795265022ULL
    };

    for (u64 a : bases)
    {
        // a 是 n 的倍数时，这一轮没有判别能力，直接跳过。
        if (a % n == 0)
        {
            continue;
        }

        // Miller-Rabin 首先检查  $a^d \bmod n$ 。

```

```

u64 x = pow_mod(a % n, d, n);
if (x == 1 || x == n - 1)
{
    continue;
}

// 连续平方；若始终到不了 n - 1，则 a 是“合数证人”。
bool witness = true;
for (int r = 1; r < s; ++r)
{
    x = mul_mod(x, x, n);
    if (x == n - 1)
    {
        witness = false;
        break;
    }
}
if (witness)
{
    return false;
}
}
return true;
}

// 随机起点和常数可以降低 Pollard-Rho 卡在坏循环上的概率。
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

u64 pollard_rho(u64 n)
{
    // 最常见的小因子直接返回，避免进入随机过程。
    if (n % 2 == 0)
    {
        return 2;
    }
    if (n % 3 == 0)
    {
        return 3;
    }

    while (true)
    {
        uniform_int_distribution<u64> pick(2, n - 2);
        u64 x = pick(rng); // 慢指针。
        u64 y = x;          // 快指针。
        u64 c = pick(rng); // 多项式  $f(v) = v^2 + c$  中的常数。
        u64 d = 1;          // 当前找到的 gcd；1 表示尚未找到因子。

        auto next = [&](u64 value)
        {
            // 计算  $f(value)$ ，所有运算都在模 n 意义下进行。
            return static_cast<u64>(
                (static_cast<u128>(mul_mod(value, value, n)) + c) % n
            );
        };
    }
}

```

```

    );
};

while (d == 1)
{
    // Floyd 判环: x 每次走一步, y 每次走两步。
    x = next(x);
    y = next(next(y));
    u64 diff = x > y ? x - y : y - x;
    // 若两个状态在某个质因子意义下相等, gcd 会暴露该因子。
    d = gcd(diff, n);
}

// d == n 表示本轮失败, 重新随机参数再试。
if (d != n)
{
    return d;
}
}

}

void factor_dfs(u64 n, vector<u64>& factors)
{
    if (n == 1)
    {
        return;
    }
    if (is_prime(n))
    {
        // 递归终点: n 已经不能继续分解。
        factors.push_back(n);
        return;
    }

    // 找到一个非平凡因子后, 把问题拆成 d 与 n / d 两部分。
    u64 d = pollard_rho(n);
    factor_dfs(d, factors);
    factor_dfs(n / d, factors);
}

vector<u64> factorize(u64 n)
{
    vector<u64> factors;
    factor_dfs(n, factors);
    // Pollard-Rho 找因子的顺序不固定, 统一排序便于使用。
    sort(factors.begin(), factors.end());
    return factors;
}

```

1.2 快速幂

计算 $a^b \bmod \text{mod}$ ，要求 $b \geq 0$ 且 $\text{mod} > 0$ 。使用 `i128` 完成中间乘法，适用于模数不超过有符号 64 位范围的场景。

```
using ll = long long;
using i128 = __int128_t;

ll power_mod(ll a, ll b, ll mod)
{
    // C++ 的负数取模仍可能为负，先把底数规范到 [0, mod)。
    a %= mod;
    if (a < 0)
    {
        a += mod;
    }

    ll res = 1 % mod; // mod == 1 时答案应为 0。
    while (b > 0)
    {
        if (b & 1)
        {
            // 中间乘积使用 i128，避免 long long 溢出。
            res = static_cast<ll>(static_cast<i128>(res) * a % mod);
        }
        a = static_cast<ll>(static_cast<i128>(a) * a % mod);
        b >>= 1;
    }
    return res;
}
```

1.3 线性筛

线性筛在 $O(n)$ 时间内得到不超过 n 的全部素数，并记录每个正整数的最小质因子，空间复杂度为 $O(n)$ 。

⚠ Warning

`vector<int> min_prime` 每个位置占 4 字节。在洛谷 P3383 的 $n = 10^8$ 上，本实现实测峰值内存约为 **416 MB**；若只需要素数表或素性标记，应改用更节省内存的筛法。

```
struct LinearSieve
{
    vector<int> min_prime; // min_prime[x]: x 的最小质因子。
    vector<int> primes;    // 按从小到大保存已经找到的全部质数。

    explicit LinearSieve(int n) : min_prime(n + 1, 0)
    {
        for (int i = 2; i <= n; ++i)
        {
            if (min_prime[i] == 0)
            {
                // i 没被更小的质数筛掉，因此 i 本身是质数。
                min_prime[i] = i;
            }
        }
    }
};
```

```

        primes.push_back(i);
    }

    for (int p : primes)
    {
        // p > min_prime[i] 时继续枚举会造成重复筛除。
        // 第二个条件保证 i * p 不越过预处理上界。
        if (p > min_prime[i] || 1LL * i * p > n)
        {
            break;
        }
        min_prime[i * p] = p;
    }
}

bool is_prime(int x) const
{
    // 质数的最小质因子就是它自己。
    return x >= 2 && min_prime[x] == x;
}
};

```

1.4 扩展欧几里得

`exgcd(a, b, x, y)` 求出一组整数解，使 $a * x + b * y = \gcd(a, b)$ 。进一步可解线性同余方程 $a * x = b \pmod{\text{mod}}$ ；返回值中的第二项是解的周期。

```

using ll = long long;
using i128 = __int128_t;

ll exgcd(ll a, ll b, ll& x, ll& y)
{
    if (b == 0)
    {
        // a * 1 + 0 * 0 = a, 递归从这里开始回代。
        x = 1;
        y = 0;
        return a;
    }

    // 递归参数中的 x、y 交换，是为了直接接住下一层的系数。
    ll g = exgcd(b, a % b, y, x);
    // 根据 a % b = a - floor(a / b) * b 回代系数。
    y -= a / b * x;
    return g;
}

optional<pair<ll, ll>> solve_linear_congruence(ll a, ll b, ll mod)
{
    assert(mod > 0);
}

```

```

11 x, y;
11 g = exgcd(a, mod, x, y);
// 统一让 gcd 为正，便于后续整除和取模。
if (g < 0)
{
    g = -g;
    x = -x;
    y = -y;
}
if (b % g != 0)
{
    // gcd(a, mod) 不能整除 b 时，同余方程无解。
    return nullopt;
}

11 period = mod / g; // 所有解按 period 循环出现。
// x * (b / g) 可能超过 64 位，因此使用 i128 计算。
11 x0 = static_cast<ll>(static_cast<i128>(x) * (b / g) % period);
if (x0 < 0)
{
    x0 += period;
}
return pair{x0, period};
}

```

1.5 模逆元

当且仅当 $\gcd(a, \text{mod}) = 1$ 时， a 在模 mod 意义下存在逆元。扩展欧几里得适用于任意正模数；费马小定理版本只适用于质数模数且 a 不能被模数整除。

```

using ll = long long;

optional<ll> inverse_mod(ll a, ll mod)
{
    assert(mod > 0);
    a %= mod;
    if (a < 0)
    {
        a += mod;
    }

    11 x, y;
    11 g = exgcd(a, mod, x, y);
    if (g != 1)
    {
        // 只有 gcd(a, mod) == 1 时逆元才存在。
        return nullopt;
    }

    // exgcd 给出的 x 可能为负，将其规范到 [0, mod)。
    x %= mod;
    if (x < 0)

```

```

    {
        x += mod;
    }
    return x;
}

11 inverse_prime_mod(11 a, 11 prime_mod)
{
    // 费马小定理:  $a^{(p-1)} \equiv 1 \pmod{p}$ , 所以逆元是  $a^{(p-2)}$ 。
    assert(a % prime_mod != 0);
    return power_mod(a, prime_mod - 2, prime_mod);
}

```

1.6 卡特兰数

卡特兰数常见于合法括号序列、不同形态的二叉树、凸多边形三角剖分等计数问题。前几项为 1, 1, 2, 5, 14, 42, ...。

常用闭式为 $C[n] = \text{binom}(2n, n) / (n + 1)$, 也可写成 $\text{binom}(2n, n) - \text{binom}(2n, n - 1)$ 。

递推式

$$C[n] = C[n - 1] * (4n - 2) / (n + 1)$$

中的除法一定整除。卡特兰数增长很快: $C[36]$ 是最后一个能放入 `unsigned long long` 的卡特兰数, $C[37]$ 已经溢出。

下面的无依赖版本会在乘法前约分, 避免“最终答案未溢出, 但中间乘积先溢出”。它已在洛谷 P1044 (该题只需要 $n \leq 18$) 通过。

```

using u64 = unsigned long long;

vector<u64> catalan_u64(int n)
{
    assert(0 <= n && n <= 36);
    vector<u64> catalan(n + 1);
    catalan[0] = 1; // 空结构也算一种方案。

    for (int i = 1; i <= n; ++i)
    {
        u64 numerator = 4ULL * i - 2;
        u64 denominator = i + 1;
        u64 previous = catalan[i - 1];

        // 分母可分别与乘数、上一项约分, 最终一定被完全消去。
        u64 common = gcd(numerator, denominator);
        numerator /= common;
        denominator /= common;

        common = gcd(previous, denominator);
        previous /= common;
        denominator /= common;
    }
}

```

```

        assert(denominator == 1);
        assert(previous <= numeric_limits<u64>::max() / numerator);
        catalan[i] = previous * numerator;
    }
    return catalan;
}

```

若需要 `n > 36` 的精确值，可使用 `Boost.Multiprecision`。2026-07-25 的洛谷 C++17 评测环境未提供该头文件，因此比赛前应确认目标平台是否支持 `boost/multiprecision/cpp_int.hpp`。

```

#include <boost/multiprecision/cpp_int.hpp>

using boost::multiprecision::cpp_int;

vector<cpp_int> catalan_exact(int n)
{
    assert(n >= 0);
    vector<cpp_int> catalan(n + 1);
    catalan[0] = 1; // 空结构也算一种方案。

    for (int i = 1; i <= n; ++i)
    {
        // 先乘后除；根据卡特兰数递推式，这里不会产生小数。
        catalan[i] = catalan[i - 1] * (4LL * i - 2) / (i + 1);
    }
    return catalan;
}

```

1.7 循环引理

若整数序列的总和大于 0，从“最后一个最小前缀和”的后一位开始循环，可得到一个所有非空前缀和都严格大于 0 的循环移位。

经典循环引理还说明：当每一项都是 `+1` 或 `-1`，且总和为 `k > 0` 时，恰有 `k` 个循环移位满足所有非空前缀和严格为正。

```

using ll = long long;

int positive_rotation_start(const vector<ll>& a)
{
    if (a.empty())
    {
        return 0;
    }

    ll total = accumulate(a.begin(), a.end(), 0LL);
    if (total <= 0)
    {
        // 总和不为正时，不保证存在满足条件的循环移位。
        return -1;
    }
}

```

```

    ll prefix = 0;
    ll minimum_prefix = 0;
    int start = 0;

    for (int i = 0; i < static_cast<int>(a.size()); ++i)
    {
        prefix += a[i];
        if (prefix <= minimum_prefix)
        {
            // 使用 <=, 确保记录的是“最后一次”取得最小值的位置。
            minimum_prefix = prefix;
            start = i + 1;
        }
    }

    // start == a.size() 表示从原序列的 0 号位置开始。
    return start % static_cast<int>(a.size());
}

```

1.8 莫比乌斯反演入门

莫比乌斯函数满足：

- $\mu(1) = 1$;
- 含有平方质因子的数， $\mu(n) = 0$;
- 其余情况下，若 n 含有 k 个不同质因子，则 $\mu(n) = (-1)^k$ 。

若

$F(n) = \sum(f(d))$ ，其中 d 枚举 n 的所有正因数，

则莫比乌斯反演给出

$f(n) = \sum(\mu(d) * F(n / d))$ 。

```

using ll = long long;

struct MobiusSieve
{
    vector<int> mu;
    vector<int> primes;
    vector<char> is_composite;

    explicit MobiusSieve(int n)
        : mu(n + 1, 0), is_composite(n + 1, false)
    {
        assert(n >= 1);
        mu[1] = 1;

        for (int i = 2; i <= n; ++i)
        {
            if (!is_composite[i])
            {

```

```

        // 质数只有一个不同质因子，所以  $\mu[p] = -1$ 。
        primes.push_back(i);
        mu[i] = -1;
    }

    for (int p : primes)
    {
        if (1LL * i * p > n)
        {
            break;
        }

        int value = i * p;
        is_composite[value] = true;
        if (i % p == 0)
        {
            // value 含有  $p^2$ ，莫比乌斯函数为 0。
            mu[value] = 0;
            break;
        }

        // 新增了一个不同质因子，符号翻转。
        mu[value] = -mu[i];
    }
}

};

vector<ll> mobius_invert(const vector<ll>& divisor_sum)
{
    // 输入约定:  $\text{divisor\_sum}[n] = \sum_{d|n} \text{original}[d]$ 。
    int n = static_cast<int>(divisor_sum.size()) - 1;
    MobiusSieve sieve(n);
    vector<ll> original(n + 1, 0);

    for (int d = 1; d <= n; ++d)
    {
        if (sieve.mu[d] == 0)
        {
            continue;
        }
        for (int multiple = d; multiple <= n; multiple += d)
        {
            original[multiple] += 1LL * sieve.mu[d]
                                * divisor_sum[multiple / d];
        }
    }
    return original;
}

ll count_coprime_pairs(ll n, ll m, const vector<int>& mu)
{
    // 统计  $1 \leq x \leq n, 1 \leq y \leq m$  且  $\gcd(x, y) = 1$  的有序数对。

```

```

int limit = static_cast<int>(min(n, m));
assert(static_cast<int>(mu.size()) > limit);
ll answer = 0;
for (int d = 1; d <= limit; ++d)
{
    answer += 1LL * mu[d] * (n / d) * (m / d);
}
return answer;
}

```

2. 数据结构

2.1 树状数组

下标采用 **1-based**。单点增加与前缀和查询均为 $O(\log n)$ ；`lower_bound` 仅在所有增量非负、前缀和单调时使用。

```

template <class T>
struct Fenwick
{
    int n;          // 可用下标范围为 [1, n]。
    vector<T> bit; // bit[i] 维护长度为 lowbit(i) 的一段后缀和。

    explicit Fenwick(int n_) : n(n_), bit(n_ + 1, T{})
    {
    }

    void add(int idx, T delta)
    {
        // 每次跳到包含 idx 的更大树状数组节点。
        for (int i = idx; i <= n; i += i & -i)
        {
            bit[i] += delta;
        }
    }

    T prefix_sum(int idx) const
    {
        T res{};
        // 不断删掉最低位的 1，把 [1, idx] 拆成若干互不相交区间。
        for (int i = idx; i > 0; i -= i & -i)
        {
            res += bit[i];
        }
        return res;
    }

    T range_sum(int left, int right) const
    {
        if (left > right)
        {
            return T{};
        }
    }
}

```

```

    }
    // [left, right] = [1, right] - [1, left - 1]。
    return prefix_sum(right) - prefix_sum(left - 1);
}

int lower_bound(T target) const
{
    // 查找最小的 idx, 使 prefix_sum(idx) >= target。
    assert(target > T{});

    int idx = 0;    // 当前已确认“前缀和仍小于 target”的最大位置。
    T current{};    // prefix_sum(idx)。
    int step = 1;
    // 从不超过 n 的最高二进制位开始试跳。
    while ((step << 1) <= n)
    {
        step <<= 1;
    }

    for (; step > 0; step >>= 1)
    {
        int next = idx + step;
        if (next <= n && current + bit[next] < target)
        {
            // 加上这一整块后仍不够 target, 可以安全跳过去。
            idx = next;
            current += bit[next];
        }
    }
    return idx + 1;
}
};

```

2.2 线段树

该版本维护 **1-based** 数组，支持区间赋值与区间求和，单次操作复杂度为 $O(\log n)$ 。`has_lazy` 用于区分“没有懒标记”和“整段赋值为 0”。

```

using ll = long long;

struct SegTreeRangeSet
{
    int n;
    vector<ll> tree;    // tree[node]: 当前区间的元素和。
    vector<ll> lazy;    // lazy[node]: 待下传的“整段赋值”。
    vector<char> has_lazy; // 单独记录是否有标记，因为赋值为 0 也是合法操作。

    explicit SegTreeRangeSet(const vector<ll>& a)
        : n(static_cast<int>(a.size()) - 1),
          tree(4 * n + 4, 0),
          lazy(4 * n + 4, 0),
          has_lazy(4 * n + 4, false)

```

```

{
    assert(n >= 1);
    build(1, 1, n, a);
}

void build(int node, int left, int right, const vector<ll>& a)
{
    if (left == right)
    {
        // 叶子节点只对应原数组中的一个位置。
        tree[node] = a[left];
        return;
    }

    int mid = (left + right) / 2;
    build(node * 2, left, mid, a);
    build(node * 2 + 1, mid + 1, right, a);
    pull(node);
}

void pull(int node)
{
    // 父区间的和等于左右子区间之和。
    tree[node] = tree[node * 2] + tree[node * 2 + 1];
}

void apply(int node, int left, int right, ll value)
{
    // 整段都变为 value, 区间和就是 value * 区间长度。
    tree[node] = value * (right - left + 1);
    lazy[node] = value;
    has_lazy[node] = true;
}

void push(int node, int left, int right)
{
    // 没有标记或已经是叶子时, 无需下传。
    if (!has_lazy[node] || left == right)
    {
        return;
    }

    int mid = (left + right) / 2;
    // 左右孩子都继承父节点的整段赋值。
    apply(node * 2, left, mid, lazy[node]);
    apply(node * 2 + 1, mid + 1, right, lazy[node]);
    has_lazy[node] = false;
}

void range_set(int query_left, int query_right, ll value)
{
    range_set(1, 1, n, query_left, query_right, value);
}

```

```

void range_set(int node, int left, int right, int query_left, int query_right, ll value)
{
    // 当前区间与修改区间完全不相交。
    if (query_right < left || right < query_left)
    {
        return;
    }
    if (query_left <= left && right <= query_right)
    {
        // 当前区间被完整覆盖，直接打懒标记，不再递归。
        apply(node, left, right, value);
        return;
    }

    // 继续访问孩子前，必须先把旧标记下传。
    push(node, left, right);
    int mid = (left + right) / 2;
    range_set(node * 2, left, mid, query_left, query_right, value);
    range_set(node * 2 + 1, mid + 1, right, query_left, query_right, value);
    pull(node);
}

ll range_sum(int query_left, int query_right)
{
    return range_sum(1, 1, n, query_left, query_right);
}

ll range_sum(int node, int left, int right, int query_left, int query_right)
{
    if (query_right < left || right < query_left)
    {
        return 0;
    }
    if (query_left <= left && right <= query_right)
    {
        // 完整覆盖时直接使用已经维护好的区间和。
        return tree[node];
    }

    push(node, left, right);
    int mid = (left + right) / 2;
    return range_sum(node * 2, left, mid, query_left, query_right)
        + range_sum(node * 2 + 1, mid + 1, right, query_left, query_right);
}

};

```

2.3 并查集

下标采用 **1-based**。路径压缩配合按集合大小合并，单次操作的均摊复杂度接近 $O(1)$ ；`merge` 返回是否真的合并了两个集合。

```
struct DSU
{
    vector<int> parent; // parent[x]: x 在并查集森林中的父节点。
    vector<int> size;   // 只保证根节点的 size 有意义。
    int set_count;      // 当前互不相交集合的数量。

    explicit DSU(int n) : parent(n + 1), size(n + 1, 1), set_count(n)
    {
        // 初始时每个点都是自己的根。
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x)
    {
        if (parent[x] == x)
        {
            return x;
        }
        // 路径压缩：以后再查询 x 时可以直接接近根节点。
        return parent[x] = find(parent[x]);
    }

    bool merge(int x, int y)
    {
        x = find(x);
        y = find(y);
        if (x == y)
        {
            return false;
        }
        if (size[x] < size[y])
        {
            // 始终把小树接到大树下面，控制树高。
            swap(x, y);
        }

        parent[y] = x; // y 不再是根，整个集合并入 x。
        size[x] += size[y];
        --set_count;
        return true;
    }

    bool same(int x, int y)
    {
        return find(x) == find(y);
    }
}
```

```
int component_size(int x)
{
    return size[find(x)];
}

};
```

2.4 分数类

分子、分母始终约分，且分母保持为正。四则运算使用 `i128` 计算中间值，并断言最终结果能放入 `ll`；除法还要求除数非零。

```
using ll = long long;
using i128 = __int128_t;

struct Fraction
{
    ll num = 0; // 分子。
    ll den = 1; // 分母始终为正。

    Fraction() = default;

    Fraction(ll num_, ll den_ = 1) : num(num_), den(den_)
    {
        normalize();
    }

    static i128 gcd_i128(i128 a, i128 b)
    {
        // 独立实现 128 位 gcd，避免中间数退回 64 位。
        if (a < 0)
        {
            a = -a;
        }
        if (b < 0)
        {
            b = -b;
        }
        while (b != 0)
        {
            a %= b;
            swap(a, b);
        }
        return a;
    }

    void normalize()
    {
        assert(den != 0);
        // 提升到 i128 后再处理 LLONG_MIN 取反等边界。
        i128 wide_num = num;
        i128 wide_den = den;
        if (wide_den < 0)
```

```

{
    // 统一把负号放到分子上。
    wide_num = -wide_num;
    wide_den = -wide_den;
}

// 构造后立刻约分，保证相等分数具有唯一表示。
i128 g = gcd_i128(wide_num, wide_den);
wide_num /= g;
wide_den /= g;
assert(wide_num >= numeric_limits<ll>::min()
    && wide_num <= numeric_limits<ll>::max());
assert(wide_den >= 1 && wide_den <= numeric_limits<ll>::max());
num = static_cast<ll>(wide_num);
den = static_cast<ll>(wide_den);
}

static Fraction from_i128(i128 num, i128 den)
{
    // 所有四则运算都先在 i128 中完成，再从这里统一约分和检查范围。
    assert(den != 0);
    if (den < 0)
    {
        num = -num;
        den = -den;
    }

    i128 g = gcd_i128(num, den);
    num /= g;
    den /= g;
    assert(num >= numeric_limits<ll>::min() && num <= numeric_limits<ll>::max());
    assert(den >= 1 && den <= numeric_limits<ll>::max());

    Fraction result;
    result.num = static_cast<ll>(num);
    result.den = static_cast<ll>(den);
    return result;
}

friend Fraction operator+(const Fraction& lhs, const Fraction& rhs)
{
    //  $a / b + c / d = (ad + cb) / bd$ 。
    return from_i128(
        static_cast<i128>(lhs.num) * rhs.den + static_cast<i128>(rhs.num) * lhs.den,
        static_cast<i128>(lhs.den) * rhs.den
    );
}

friend Fraction operator-(const Fraction& lhs, const Fraction& rhs)
{
    return from_i128(
        static_cast<i128>(lhs.num) * rhs.den - static_cast<i128>(rhs.num) * lhs.den,
        static_cast<i128>(lhs.den) * rhs.den
    );
}

```

```

    );
}

friend Fraction operator*(const Fraction& lhs, const Fraction& rhs)
{
    // 中间乘积可能超过 11, 因此先转换为 i128。
    return from_i128(
        static_cast<i128>(lhs.num) * rhs.num,
        static_cast<i128>(lhs.den) * rhs.den
    );
}

friend Fraction operator/(const Fraction& lhs, const Fraction& rhs)
{
    // 除以 c / d 等价于乘以 d / c, 因此 c 不能为 0。
    assert(rhs.num != 0);
    i128 num = static_cast<i128>(lhs.num) * rhs.den;
    i128 den = static_cast<i128>(lhs.den) * rhs.num;
    if (den < 0)
    {
        num = -num;
        den = -den;
    }
    return from_i128(num, den);
}

friend bool operator==(const Fraction& lhs, const Fraction& rhs)
{
    return lhs.num == rhs.num && lhs.den == rhs.den;
}

friend bool operator<(const Fraction& lhs, const Fraction& rhs)
{
    // 分母均为正, 可以安全交叉相乘比较。
    return static_cast<i128>(lhs.num) * rhs.den
        < static_cast<i128>(rhs.num) * lhs.den;
}

friend ostream& operator<<(ostream& out, const Fraction& value)
{
    out << value.num;
    if (value.den != 1)
    {
        out << '/' << value.den;
    }
    return out;
}
};

```

2.5 取模类

该模板面向小于 2^{32} 的质数模数。乘法用 `uint64_t` 承接中间结果；除法通过费马小定理求逆元，因此除数不能为 0。

```
template <uint32_t MOD>
class ModInt
{
public:
    ModInt(long long value = 0)
    {
        // 任何输入都规范到 [0, MOD)。
        value %= static_cast<long long>(MOD);
        if (value < 0)
        {
            value += MOD;
        }
        value_ = static_cast<uint32_t>(value);
    }

    uint32_t value() const
    {
        return value_;
    }

    ModInt power(uint64_t exponent) const
    {
        // 与普通快速幂相同，只是每次乘法都会自动取模。
        ModInt base = *this;
        ModInt res = 1;
        while (exponent > 0)
        {
            if (exponent & 1)
            {
                res *= base;
            }
            base *= base;
            exponent >>= 1;
        }
        return res;
    }

    ModInt inverse() const
    {
        // MOD 必须为质数；0 没有乘法逆元。
        assert(value_ != 0);
        return power(MOD - 2);
    }

    ModInt& operator+=(const ModInt& rhs)
    {
        // 两个模数内的值相加最多小于 2 * MOD，只需减一次。
```

```

        uint64_t sum = static_cast<uint64_t>(value_) + rhs.value_;
        value_ = static_cast<uint32_t>(sum >= MOD ? sum - MOD : sum);
        return *this;
    }

ModInt& operator--=(const ModInt& rhs)
{
    // 不够减时先加一个 MOD，避免无符号下溢。
    uint64_t difference = value_ >= rhs.value_
        ? value_ - rhs.value_
        : static_cast<uint64_t>(value_) + MOD - rhs.value_;
    value_ = static_cast<uint32_t>(difference);
    return *this;
}

ModInt& operator*=(const ModInt& rhs)
{
    // MOD < 2^32，因此两个余数的乘积可放入 uint64_t。
    value_ = static_cast<uint32_t>(
        static_cast<uint64_t>(value_) * rhs.value_ % MOD
    );
    return *this;
}

ModInt& operator/=(const ModInt& rhs)
{
    return *this *= rhs.inverse();
}

friend ModInt operator+(ModInt lhs, const ModInt& rhs)
{
    return lhs += rhs;
}

friend ModInt operator-(ModInt lhs, const ModInt& rhs)
{
    return lhs -= rhs;
}

friend ModInt operator*(ModInt lhs, const ModInt& rhs)
{
    return lhs *= rhs;
}

friend ModInt operator/(ModInt lhs, const ModInt& rhs)
{
    return lhs /= rhs;
}

friend bool operator==(const ModInt& lhs, const ModInt& rhs)
{
    return lhs.value_ == rhs.value_;
}

```

```

    friend ostream& operator<<(ostream& out, const ModInt& value)
    {
        return out << value.value_;
    }

private:
    uint32_t value_ = 0;
};

using Z = ModInt<998244353>;

```

2.6 自定义 tuple 排序

一次性比较规则优先使用无捕获 lambda。比较器必须满足严格弱序；当前示例先按整数升序，再按字符串字典序升序。

```

using Item = tuple<int, string>;

void sort_items(vector<Item>& items)
{
    sort(items.begin(), items.end(), [](const Item& lhs, const Item& rhs)
    {
        // 第一关键字不同，按整数升序。
        if (get<0>(lhs) != get<0>(rhs))
        {
            return get<0>(lhs) < get<0>(rhs);
        }
        // 第一关键字相同时，再按字符串升序。
        return get<1>(lhs) < get<1>(rhs);
    });
}

```

2.7 自定义优先队列（函数对象）

`priority_queue` 的比较器在 `lhs` 优先级低于 `rhs` 时返回 `true`。示例中分数越高优先级越高；分数相同则年龄较小者优先。

```

struct Person
{
    string name;
    int age;
    int score;
};

struct PersonPriority
{
    bool operator()(const Person& lhs, const Person& rhs) const
    {
        // priority_queue 中返回 true 表示 lhs 应排在 rhs 后面。
        if (lhs.score != rhs.score)

```

```

    {
        return lhs.score < rhs.score;
    }
    return lhs.age > rhs.age;
}

};

priority_queue<Person, vector<Person>, PersonPriority> people;

```

2.8 自定义优先队列 (lambda)

比较规则只使用一次时，可通过 `decltype` 把 lambda 类型传给容器。

```

auto person_priority = [](const Person& lhs, const Person& rhs)
{
    // 分数越高越优先；分数相同时，年龄越小越优先。
    if (lhs.score != rhs.score)
    {
        return lhs.score < rhs.score;
    }
    return lhs.age > rhs.age;
};

priority_queue<Person, vector<Person>, decltype(person_priority)> people(person_priority);

```

2.9 自定义集合 (函数对象)

`set` 会把“互不小于”的两个元素视为等价。若需要保留同龄但姓名不同的对象，比较器必须增加姓名作为次关键字；允许完全重复时改用 `multiset`。

```

struct PersonOrder
{
    bool operator()(const Person& lhs, const Person& rhs) const
    {
        // set 的比较器必须形成严格弱序，不能在相等时返回 true。
        if (lhs.age != rhs.age)
        {
            return lhs.age < rhs.age;
        }
        return lhs.name < rhs.name;
    }
};

set<Person, PersonOrder> people;
multiset<Person, PersonOrder> people_with_duplicates;

```

2.10 自定义集合 (lambda)

```
auto person_order = [] (const Person& lhs, const Person& rhs)
{
    // set 先按年龄、再按姓名建立唯一的升序关系。
    if (lhs.age != rhs.age)
    {
        return lhs.age < rhs.age;
    }
    return lhs.name < rhs.name;
};

set<Person, decltype(person_order)> people(person_order);
multiset<Person, decltype(person_order)> people_with_duplicates(person_order);
```

2.11 异或线性基

线性基维护一组彼此线性无关的二进制向量，使原集合任意子集的异或和都能由它们表示。常用于最大子集异或和、判断某个值能否被表示、求异或空间维数。

下面处理 `uint64_t`，从最高位向最低位消元。

```
using u64 = uint64_t;

struct XorBasis
{
    static constexpr int BITS = 64;
    array<u64, BITS> basis{};
    int rank = 0; // 当前线性无关向量的数量。

    bool insert(u64 value)
    {
        for (int bit = BITS - 1; bit >= 0; --bit)
        {
            if (((value >> bit) & 1ULL) == 0)
            {
                continue;
            }

            if (basis[bit] == 0)
            {
                // value 的最高位此前没有主元，可以加入线性基。
                basis[bit] = value;
                ++rank;
                return true;
            }

            // 消掉 value 的当前最高位，继续处理更低位。
            value ^= basis[bit];
        }

        // value 被完全消为 0，说明它能由已有向量异或得到。
    }
};
```

```

        return false;
    }

    bool can_represent(u64 value) const
    {
        for (int bit = BITS - 1; bit >= 0; --bit)
        {
            if (((value >> bit) & 1ULL) != 0)
            {
                value ^= basis[bit];
            }
        }
        return value == 0;
    }

    u64 maximum_xor(u64 initial = 0) const
    {
        u64 answer = initial;
        for (int bit = BITS - 1; bit >= 0; --bit)
        {
            // 若异或当前主元能让数值变大，就选用它。
            answer = max(answer, answer ^ basis[bit]);
        }
        return answer;
    }
};

```

3. 图论

3.1 邻接矩阵

邻接矩阵适合点数较少的稠密图，空间复杂度为 $O(n^2)$ 。以下模板使用 **0-based** 点编号；重边保留最小边权。

```

using ll = long long;

const ll INF = numeric_limits<ll>::max() / 4;

vector<vector<ll>> make_matrix(int n)
{
    // 初始时任意两个不同点都不可达。
    vector<vector<ll>> graph(n, vector<ll>(n, INF));
    for (int i = 0; i < n; ++i)
    {
        // 点到自身的最短距离为 0。
        graph[i][i] = 0;
    }
    return graph;
}

void add_directed_edge(vector<vector<ll>>& graph, int from, int to, ll weight)
{
    // 出现重边时只保留最小边权。

```

```

        graph[from][to] = min(graph[from][to], weight);
    }

    void add_undirected_edge(vector<vector<ll>>& graph, int u, int v, ll weight)
    {
        add_directed_edge(graph, u, v, weight);
        add_directed_edge(graph, v, u, weight);
    }

```

3.2 邻接表

邻接表适合竞赛中的大多数稀疏图。以下模板使用 **1-based** 点编号，边记录终点与权值。

```

using ll = long long;

struct Edge
{
    int to;          // 边的终点。
    ll weight;       // 边权。
};

using Graph = vector<vector<Edge>>;

void add_directed_edge(Graph& graph, int from, int to, ll weight = 1)
{
    graph[from].push_back({to, weight});
}

void add_undirected_edge(Graph& graph, int u, int v, ll weight = 1)
{
    // 无向边等价于两条方向相反、权值相同的有向边。
    add_directed_edge(graph, u, v, weight);
    add_directed_edge(graph, v, u, weight);
}

```

3.3 DFS 深度优先搜索

时间复杂度为 $O(n + m)$ 。递归版本简洁，但链状图很深时可能栈溢出，此时应改用显式栈。

```

void dfs(int u, const Graph& graph, vector<char>& visited)
{
    // 进入节点后立刻标记，避免在有环图中重复递归。
    visited[u] = true;
    for (const Edge& edge : graph[u])
    {
        if (!visited[edge.to])
        {
            // 沿尚未访问的边继续深入。
            dfs(edge.to, graph, visited);
        }
    }
}

```

3.4 BFS 广度优先搜索

无权图中，BFS 可在 $O(n + m)$ 时间内求起点到所有可达点的最少边数。不可达点的距离保持为 -1 。

```

vector<int> bfs(int start, const Graph& graph)
{
    int n = static_cast<int>(graph.size()) - 1;
    vector<int> dist(n + 1, -1);
    queue<int> q;

    dist[start] = 0; // 起点到自身需要经过 0 条边。
    q.push(start);

    while (!q.empty())
    {
        int u = q.front();
        q.pop();

        for (const Edge& edge : graph[u])
        {
            int v = edge.to;
            if (dist[v] != -1)
            {
                // BFS 第一次到达一个点时，距离已经最短。
                continue;
            }
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
    return dist;
}

```

3.5 Dijkstra 最短路

适用于所有边权非负的图，使用二叉堆时复杂度为 $O((n + m) \log n)$ 。弹出旧状态时直接跳过，不需要额外的 `visited` 数组。

```
using ll = long long;

vector<ll> dijkstra(int start, const Graph& graph)
{
    const ll INF = numeric_limits<ll>::max() / 4;
    int n = static_cast<int>(graph.size()) - 1;
    vector<ll> dist(n + 1, INF);

    // first 是当前距离, second 是点编号; greater 形成小根堆。
    using State = pair<ll, int>;
    priority_queue<State, vector<State>, greater<State>> pq;

    dist[start] = 0;
    pq.push({0, start});

    while (!pq.empty())
    {
        auto [current_dist, u] = pq.top();
        pq.pop();

        if (current_dist != dist[u])
        {
            // u 之前已经被更短的路径更新; 这是留在堆中的旧状态。
            continue;
        }

        for (const Edge& edge : graph[u])
        {
            assert(edge.weight >= 0);
            if (dist[edge.to] > current_dist + edge.weight)
            {
                // 松弛: 经过 u 到达 edge.to 得到了更短路径。
                dist[edge.to] = current_dist + edge.weight;
                pq.push({dist[edge.to], edge.to});
            }
        }
    }
    return dist;
}
```

3.6 SPFA 与负环检测

SPFA 能处理负边，并检测从起点可达的负环；若存在负环则返回 `nullopt`。其最坏复杂度为 $O(nm)$ ，没有负边时优先使用 Dijkstra。

⚠ Caution

返回结果只覆盖 `start` 可达的点。洛谷 P3385 只要求判断从 1 号点可达的负环，因此应调用 `spfa(1, graph)`；不要无条件添加超级源，否则会把不连通分量中的负环误判为答案。

```
using ll = long long;

optional<vector<ll>> spfa(int start, const Graph& graph)
{
    const ll INF = numeric_limits<ll>::max() / 4;
    int n = static_cast<int>(graph.size()) - 1;

    vector<ll> dist(n + 1, INF);
    // path_edges[v]: 当前用于更新 dist[v] 的路径包含多少条边。
    vector<int> path_edges(n + 1, 0);
    // 同一个点不重复入队，避免队列中出现大量相同状态。
    vector<char> in_queue(n + 1, false);
    queue<int> q;

    dist[start] = 0;
    in_queue[start] = true;
    q.push(start);

    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        // 出队后允许 u 在未来再次被更新并入队。
        in_queue[u] = false;

        for (const Edge& edge : graph[u])
        {
            int v = edge.to;
            if (dist[v] <= dist[u] + edge.weight)
            {
                // 经过 u 不能改善 v，跳过这条边。
                continue;
            }

            dist[v] = dist[u] + edge.weight;
            path_edges[v] = path_edges[u] + 1;
            if (path_edges[v] >= n)
            {
                // 含至少 n 条边的严格变短路径必然重复顶点，说明存在可达负环。
                return nullopt;
            }

            if (!in_queue[v])
            {
                in_queue[v] = true;
                q.push(v);
            }
        }
    }
}
```

```

    }
    return dist;
}

```

3.7 最小生成树 (Prim)

Prim 适用于无向图。优先队列中保存“连接当前生成树与未选点的最小边”；若图不连通则返回 `nullopt`。复杂度为 $O((n + m) \log n)$ 。

```

using ll = long long;

optional<ll> prim(int start, const Graph& graph)
{
    int n = static_cast<int>(graph.size()) - 1;
    vector<char> used(n + 1, false);

    using State = pair<ll, int>;
    priority_queue<State, vector<State>, greater<State>> pq;
    // 起点通过一条权值为 0 的“虚拟边”加入生成树。
    pq.push({0, start});

    ll total_weight = 0;
    int used_count = 0;

    while (!pq.empty())
    {
        auto [weight, u] = pq.top();
        pq.pop();

        if (used[u])
        {
            // 同一个点可能由多条候选边加入堆，只采用第一次弹出的最小边。
            continue;
        }
        used[u] = true;
        total_weight += weight;
        ++used_count;

        for (const Edge& edge : graph[u])
        {
            if (!used[edge.to])
            {
                // 暂时把所有跨越割的边加入候选集合。
                pq.push({edge.weight, edge.to});
            }
        }
    }

    if (used_count != n)
    {
        // 没有访问完所有点，原图不连通，不存在生成树。
        return nullopt;
    }
}

```

```

    }
    return total_weight;
}

```

3.8 Floyd 全源最短路

Floyd 适合点数较少的稠密图，可在 $O(n^3)$ 时间和 $O(n^2)$ 空间内求任意两点间最短路。矩阵使用 **0-based** 点编号。

```

using ll = long long;

vector<vector<ll>> floyd_warshall(vector<vector<ll>> dist)
{
    const ll INF = numeric_limits<ll>::max() / 4;
    int n = static_cast<int>(dist.size());

    for (int middle = 0; middle < n; ++middle)
    {
        // 本轮允许 middle 作为路径中的中间点。
        for (int from = 0; from < n; ++from)
        {
            if (dist[from][middle] == INF)
            {
                continue;
            }
            for (int to = 0; to < n; ++to)
            {
                if (dist[middle][to] == INF)
                {
                    continue;
                }

                dist[from][to] = min(
                    dist[from][to],
                    dist[from][middle] + dist[middle][to]
                );
            }
        }
    }
    return dist;
}

```

3.9 二分图最大匹配（匈牙利算法）

左右两部分点均使用 **1-based** 编号。每次为一个左部点寻找增广路；若右部点已经匹配，就递归尝试让其原搭档改配其他点。复杂度为 $O(VE)$ 。

```

struct KuhnMatcher
{
    int left_size;
    int right_size;
}

```

```

vector<vector<int>> graph;
vector<int> match_right;
vector<int> visited_stamp;
int stamp = 0;

KuhnMatcher(int left_size_, int right_size_)
: left_size(left_size_),
  right_size(right_size_),
  graph(left_size_ + 1),
  match_right(right_size_ + 1, 0),
  visited_stamp(right_size_ + 1, 0)
{
}

void add_edge(int left, int right)
{
    graph[left].push_back(right);
}

bool augment(int left)
{
    for (int right : graph[left])
    {
        if (visited_stamp[right] == stamp)
        {
            // 本轮增广已经尝试过这个右部点。
            continue;
        }
        visited_stamp[right] = stamp;

        if (match_right[right] == 0
            || augment(match_right[right]))
        {
            // right 未匹配, 或它的原搭档成功改配到了别处。
            match_right[right] = left;
            return true;
        }
    }
    return false;
}

int maximum_matching()
{
    int matches = 0;
    for (int left = 1; left <= left_size; ++left)
    {
        // 时间戳代替每轮清空 visited 数组。
        ++stamp;
        if (augment(left))
        {
            ++matches;
        }
    }
}

```

```
        return matches;
    }
};
```

4. 128 位整数

GNU C++ 提供 `__int128_t` 和 `__uint128_t`，但标准流没有原生输入输出重载。以下转换函数支持符号并检查十进制文本是否越界。

4.1 字符串转换与输出

```
using i128 = __int128_t;
using u128 = __uint128_t;

i128 parse_i128(string_view text)
{
    assert(!text.empty());

    int pos = 0;
    bool negative = false;
    // 单独读取符号，后面只累积绝对值。
    if (text[pos] == '+' || text[pos] == '-')
    {
        negative = text[pos] == '-';
        ++pos;
    }
    assert(pos < static_cast<int>(text.size()));

    // i128 的负数绝对值上限比正数上限多 1。
    const u128 negative_limit = static_cast<u128>(1) << 127;
    const u128 positive_limit = negative_limit - 1;
    const u128 limit = negative ? negative_limit : positive_limit;

    u128 magnitude = 0;
    for (; pos < static_cast<int>(text.size()); ++pos)
    {
        assert(isdigit(static_cast<unsigned char>(text[pos])));
        int digit = text[pos] - '0';
        // 在执行 magnitude * 10 + digit 前检查是否越界。
        assert(magnitude <= (limit - digit) / 10);
        magnitude = magnitude * 10 + digit;
    }

    if (!negative)
    {
        return static_cast<i128>(magnitude);
    }
    if (magnitude == negative_limit)
    {
        // 最小 i128 不能直接写成 -static_cast<i128>(2^127)。
        return -static_cast<i128>(magnitude - 1) - 1;
    }
}
```

```

        return -static_cast<i128>(magnitude);
    }

    string to_string_i128(i128 value)
    {
        if (value == 0)
        {
            return "0";
        }

        bool negative = value < 0;
        // 用 -(value + 1) + 1 安全取得最小负数的绝对值。
        u128 magnitude = negative
            ? static_cast<u128>(-(value + 1)) + 1
            : static_cast<u128>(value);

        string text;
        while (magnitude > 0)
        {
            // 当前最低十进制位先写入，最后再整体翻转。
            text.push_back(static_cast<char>('0' + magnitude % 10));
            magnitude /= 10;
        }
        if (negative)
        {
            text.push_back('-');
        }
        reverse(text.begin(), text.end());
        return text;
    }
}

```

4.2 整数平方根

返回最大的 x ，使 $x * x \leq n$ 。比较时使用 $x \leq n / x$ ，避免在接近 128 位上界时乘法溢出。

```

using u128 = __uint128_t;

u128 sqrt_u128(u128 n)
{
    // 任意 u128 的平方根都不会超过  $2^{64} - 1$ 。
    u128 left = 0;
    u128 right = (static_cast<u128>(1) << 64) - 1;

    while (left < right)
    {
        // 取上中位数，避免 left 无法向右收缩。
        u128 mid = left + (right - left + 1) / 2;
        if (mid <= n / mid)
        {
            //  $mid^2 \leq n$ ，答案至少为 mid。
            left = mid;
        }
    }
}

```

```

        else
        {
            right = mid - 1;
        }
    }
    return left;
}

```

4.3 最大公约数

```

using u128 = __uint128_t;

u128 gcd_u128(u128 a, u128 b)
{
    while (b != 0)
    {
        // 欧几里得算法: gcd(a, b) = gcd(b, a mod b)。
        a %= b;
        swap(a, b);
    }
    return a;
}

```

5. 精确整数函数

5.1 精确平方根

`sqrtl` 先给出近似值，再用 `i128` 校正，返回 `floor(sqrt(n))`。前置条件为 `n >= 0`。

```

using ll = long long;
using i128 = __int128_t;

ll integer_sqrt(ll n)
{
    assert(n >= 0);
    // 浮点开方只负责给出接近答案的初值。
    ll root = static_cast<ll>(sqrtl(static_cast<long double>(n)));

    // 浮点误差可能让初值偏小，向上校正。
    while (static_cast<i128>(root + 1) * (root + 1) <= n)
    {
        ++root;
    }
    // 浮点误差也可能让初值偏大，向下校正。
    while (static_cast<i128>(root) * root > n)
    {
        --root;
    }
    return root;
}

```

5.2 精确对数上取整

返回最小的非负整数 k ，使 $\text{base}^k \geq \text{target}$ 。要求 $\text{base} \geq 2$ 且 $\text{target} \geq 1$ 。

```
using ll = long long;
using i128 = __int128_t;

int ceil_log(ll base, ll target)
{
    assert(base >= 2 && target >= 1);

    i128 value = 1; // 始终等于  $\text{base}^{\text{exponent}}$ 。
    int exponent = 0;
    while (value < target)
    {
        value *= base;
        ++exponent;
    }
    return exponent;
}
```

6. 字符串

6.1 前缀函数 (KMP)

前缀函数 $\text{pi}[i]$ 表示 $s[0..i]$ 的最长相等真前缀与真后缀长度。构造和匹配均为 $O(n)$ ，所有位置采用 **0-based**。

```
vector<int> prefix_function(string_view s)
{
    int n = static_cast<int>(s.size());
    vector<int> pi(n, 0);

    for (int i = 1; i < n; ++i)
    {
        // 先尝试延长前一个位置的最长 border。
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
        {
            // 当前 border 无法延长，跳到它自己的最长 border。
            j = pi[j - 1];
        }
        if (s[i] == s[j])
        {
            // 新字符匹配成功，border 长度增加 1。
            ++j;
        }
        pi[i] = j;
    }
    return pi;
}
```

```

vector<int> find_occurrences(string_view text, string_view pattern)
{
    if (pattern.empty())
    {
        return {};
    }

    vector<int> pi = prefix_function(pattern);
    vector<int> positions;
    int matched = 0; // 当前已经匹配的模式串前缀长度。

    for (int i = 0; i < static_cast<int>(text.size()); ++i)
    {
        while (matched > 0 && text[i] != pattern[matched])
        {
            // 失配时利用 pi 跳转，不需要让文本指针回退。
            matched = pi[matched - 1];
        }
        if (text[i] == pattern[matched])
        {
            ++matched;
        }
        if (matched == static_cast<int>(pattern.size()))
        {
            // 当前匹配在 i 结束，起点为 i - |pattern| + 1。
            positions.push_back(i - matched + 1);
            // 继续保留可重叠匹配的最长后缀。
            matched = pi[matched - 1];
        }
    }
    return positions;
}

```

6.2 字典树 (Trie)

该版本维护小写英文字母，根节点编号为 0，字符串位置采用 **0-based**。插入和查询复杂度均为 $O(|s|)$ ；`end_count` 可区分重复插入。

```

struct Trie
{
    struct Node
    {
        array<int, 26> next{}; // next[c]: 沿字符 c 转移到的节点编号。
        int end_count = 0;    // 有多少个完整单词在该节点结束。
    };

    vector<Node> nodes; // nodes[0] 是根节点，同时 0 也表示“没有孩子”。

    Trie() : nodes(1)
    {
    }
}

```

```

static int index(char ch)
{
    assert('a' <= ch && ch <= 'z');
    return ch - 'a';
}

void insert(string_view word)
{
    int u = 0;
    for (char ch : word)
    {
        int c = index(ch);
        if (nodes[u].next[c] == 0)
        {
            // 当前前缀不存在，创建一个新节点。
            nodes[u].next[c] = static_cast<int>(nodes.size());
            nodes.push_back({});
        }
        u = nodes[u].next[c];
    }
    // 只有走完整个单词后才增加结束计数。
    ++nodes[u].end_count;
}

int count(string_view word) const
{
    int u = 0;
    for (char ch : word)
    {
        int c = index(ch);
        if (nodes[u].next[c] == 0)
        {
            // 中途断路，说明这个完整单词从未插入。
            return 0;
        }
        u = nodes[u].next[c];
    }
    return nodes[u].end_count;
}

bool starts_with(string_view prefix) const
{
    int u = 0;
    for (char ch : prefix)
    {
        int c = index(ch);
        if (nodes[u].next[c] == 0)
        {
            return false;
        }
        u = nodes[u].next[c];
    }
    return true;
}

```

```
}  
};
```

6.3 AC 自动机

AC 自动机把多模式串匹配合并到一棵 Trie 上，并通过失败指针在失配时跳到最长可用后缀。构建复杂度与 Trie 节点数乘字符集大小有关，扫描文本为 $O(|\text{text}|)$ 。

下面采用 `D:\ACM\模板笔记\字典树+AC自动机` 中的拓扑回传思路：扫描时只给当前状态计数，扫描结束后按 BFS 逆序把次数累加到失败指针父节点，即可得到每个模式串的出现次数。

```
using ll = long long;  
  
struct Ahocorasick  
{  
    struct Node  
    {  
        array<int, 26> next{};  
        int fail = 0;  
        ll visits = 0;  
    };  
  
    vector<Node> nodes;  
    vector<int> bfs_order;  
  
    Ahocorasick() : nodes(1)  
    {  
    }  
  
    static int index(char ch)  
    {  
        assert('a' <= ch && ch <= 'z');  
        return ch - 'a';  
    }  
  
    int insert(string_view pattern)  
    {  
        int u = 0;  
        for (char ch : pattern)  
        {  
            int c = index(ch);  
            if (nodes[u].next[c] == 0)  
            {  
                nodes[u].next[c] = static_cast<int>(nodes.size());  
                nodes.push_back({});  
            }  
            u = nodes[u].next[c];  
        }  
  
        // 保存这个终止节点，查询后可直接读取该模式串的出现次数。  
        return u;  
    }  
}
```

```

void build()
{
    queue<int> q;
    bfs_order.clear();

    // 根节点的直接孩子失败后仍回到根节点。
    for (int c = 0; c < 26; ++c)
    {
        int child = nodes[0].next[c];
        if (child != 0)
        {
            nodes[child].fail = 0;
            q.push(child);
        }
    }

    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        bfs_order.push_back(u);

        for (int c = 0; c < 26; ++c)
        {
            int child = nodes[u].next[c];
            if (child != 0)
            {
                // u --c--> child 失配时, 转到 fail[u] 再走字符 c。
                nodes[child].fail = nodes[nodes[u].fail].next[c];
                q.push(child);
            }
            else
            {
                // 补全 Trie 图, 扫描文本时每个字符只需一次转移。
                nodes[u].next[c] = nodes[nodes[u].fail].next[c];
            }
        }
    }
}

void scan(string_view text)
{
    int u = 0;
    for (char ch : text)
    {
        u = nodes[u].next[index(ch)];
        ++nodes[u].visits;
    }
}

void propagate_counts()
{

```

```

        // 子节点对应更长的字符串，必须先把它次数传给较短后缀。
        for (int i = static_cast<int>(bfs_order.size()) - 1; i >= 0; --i)
        {
            int u = bfs_order[i];
            nodes[nodes[u].fail].visits += nodes[u].visits;
        }
    }

    ll occurrences(int terminal_node) const
    {
        return nodes[terminal_node].visits;
    }

    void clear_counts()
    {
        // 复用自动机匹配另一批文本前，清空上一次的统计值。
        for (Node& node : nodes)
        {
            node.visits = 0;
        }
    }
};

vector<ll> count_patterns(
    const vector<string>& patterns,
    string_view text
)
{
    AhoCorasick automaton;
    vector<int> terminal_nodes;
    terminal_nodes.reserve(patterns.size());

    for (const string& pattern : patterns)
    {
        terminal_nodes.push_back(automaton.insert(pattern));
    }

    automaton.build();
    automaton.scan(text);
    automaton.propagate_counts();

    vector<ll> answer;
    answer.reserve(patterns.size());
    for (int terminal : terminal_nodes)
    {
        answer.push_back(automaton.occurrences(terminal));
    }
    return answer;
}

```

7. 实用工具

7.1 作用域计时器

对象离开作用域时自动输出耗时，适合配合数据生成器比较不同实现。计时结果写入 `cerr`，不会污染正常答案输出。

```
class ScopedTimer
{
public:
    explicit ScopedTimer(string label_ = "elapsed")
        : label(move(label_)), start(chrono::steady_clock::now())
    {
        // 构造对象时自动记录起点，无需手动调用 start()。
    }

    ~ScopedTimer()
    {
        // RAII: 对象离开作用域时自动计算并输出耗时。
        auto finish = chrono::steady_clock::now();
        double milliseconds = chrono::duration<double, milli>(finish - start).count();
        cerr << label << ": " << fixed << setprecision(3)
            << milliseconds << " ms\n";
    }

private:
    string label;
    chrono::steady_clock::time_point start;
};
```

7.2 随机数据生成器

`uniform_int_distribution` 的区间两端均包含在结果中。固定种子便于复现错误；使用时间种子则适合随机压力测试。

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

template <class Int>
vector<Int> random_integers(Int left, Int right, int count)
{
    // 防止误把浮点类型传入 uniform_int_distribution。
    static_assert(is_integral_v<Int>);
    assert(left <= right && count >= 0);

    // 左右端点都可能被生成。
    uniform_int_distribution<Int> distribution(left, right);
    vector<Int> values;
    values.reserve(count);

    for (int i = 0; i < count; ++i)
    {
```

```

        values.push_back(distribution(rng));
    }
    return values;
}

```

7.3 组合类型哈希

对输入先做 SplitMix64 扰动，可降低构造碰撞数据攻击 `unordered_map` / `unordered_set` 的风险。不要向 `namespace std` 添加 `tuple` 的偏特化；把自定义哈希作为容器的第三个模板参数传入。

```

uint64_t splitmix64(uint64_t x)
{
    // 多轮异或、移位和乘法把相近输入充分打散。
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

void hash_combine(size_t& seed, size_t value)
{
    // 每次程序启动使用不同盐值，降低对抗构造哈希碰撞的风险。
    static const uint64_t salt =
        chrono::steady_clock::now().time_since_epoch().count();
    seed ^= splitmix64(static_cast<uint64_t>(value) + salt + seed);
}

struct PairHash
{
    template <class First, class Second>
    size_t operator()(const pair<First, Second>& value) const
    {
        size_t seed = 0;
        // 依次把 pair 的两个成员混入同一个种子。
        hash_combine(seed, hash<First>{}(value.first));
        hash_combine(seed, hash<Second>{}(value.second));
        return seed;
    }
};

struct TupleHash
{
    template <class... Types>
    size_t operator()(const tuple<Types...>& value) const
    {
        size_t seed = 0;
        // apply 展开 tuple，折叠表达式按顺序合并每个元素的哈希。
        apply([&](const auto&... elements)
        {
            (hash_combine(
                seed,
                hash<decay_t<decltype(elements)>>{}(elements)
            ...
        }, elements);
        return seed;
    }
};

```

```

        ), ...);
    }, value);
    return seed;
}
};

unordered_set<pair<int, int>, PairHash> points;
unordered_set<tuple<int, string, double>, TupleHash> records;

```

7.4 字符串流

`istringstream` 用于解析，`ostringstream` 用于拼接。解析失败后流会进入失败状态，可直接用于循环条件。

```

string normalize_integer_list(string_view text)
{
    // istringstream 会自动跳过任意数量的空白字符。
    istringstream input{string(text)};
    vector<int> values;
    for (int value; input >> value; )
    {
        values.push_back(value);
    }

    ostringstream output;
    for (int i = 0; i < static_cast<int>(values.size()); ++i)
    {
        if (i > 0)
        {
            // 从第二个数字开始，在前面补一个空格。
            output << ' ';
        }
        output << values[i];
    }
    return output.str();
}

```

8. 动态规划

动态规划最重要的不是背公式，而是按顺序回答四个问题：

1. `dp` 状态具体表示什么；
2. 当前状态可以从哪些已知状态转移；
3. 初始状态和不可达状态如何设置；
4. 枚举顺序是否保证转移来源已经计算完成。

本章结合 `D:\ACM\dp` 与 `D:\ACM\模板笔记` 中的练习代码，整理为可复用的通用模板。

8.1 线性 DP：青蛙跳跃

`dp[i]` 表示到达第 `i` 个位置的最小代价。当前位置只依赖前面至多 `max_jump` 个状态。

```
using ll = long long;

ll minimum_frog_cost(const vector<ll>& height, int max_jump)
{
    int n = static_cast<int>(height.size());
    if (n == 0)
    {
        return 0;
    }

    const ll INF = numeric_limits<ll>::max() / 4;
    vector<ll> dp(n, INF);
    dp[0] = 0; // 已经站在起点，不需要代价。

    for (int i = 1; i < n; ++i)
    {
        for (int jump = 1; jump <= max_jump && jump <= i; ++jump)
        {
            int previous = i - jump;
            dp[i] = min(
                dp[i],
                dp[previous] + abs(height[i] - height[previous])
            );
        }
    }
    return dp[n - 1];
}
```

8.2 0/1 背包与完全背包

`dp[capacity]` 表示容量不超过 `capacity` 时能取得的最大价值。

- 0/1 背包中每件物品最多使用一次，容量必须倒序枚举；
- 完全背包中每件物品可以重复使用，容量必须正序枚举。

```
using ll = long long;

ll knapsack_01(
    const vector<int>& weight,
    const vector<ll>& value,
    int capacity
)
{
    assert(weight.size() == value.size());
    vector<ll> dp(capacity + 1, 0);

    for (int i = 0; i < static_cast<int>(weight.size()); ++i)
    {
```

```

    assert(weight[i] > 0);
    // 倒序保证 dp[current - weight[i]] 仍属于“未使用第 i 件物品”的状态。
    for (int current = capacity; current >= weight[i]; --current)
    {
        dp[current] = max(
            dp[current],
            dp[current - weight[i]] + value[i]
        );
    }
}
return dp[capacity];
}

11 complete_knapsack(
    const vector<int>& weight,
    const vector<ll>& value,
    int capacity
)
{
    assert(weight.size() == value.size());
    vector<ll> dp(capacity + 1, 0);

    for (int i = 0; i < static_cast<int>(weight.size()); ++i)
    {
        assert(weight[i] > 0);
        // 正序使更新后的状态可以继续选择同一件物品。
        for (int current = weight[i]; current <= capacity; ++current)
        {
            dp[current] = max(
                dp[current],
                dp[current - weight[i]] + value[i]
            );
        }
    }
    return dp[capacity];
}

```

8.3 按价值定义的 0/1 背包

当容量很大、总价值较小时，改用 `dp[value]` = 取得该价值所需的最小重量。这正是 AtCoder DP Contest E 的常见做法。

```

using ll = long long;

int knapsack_by_value(
    const vector<ll>& weight,
    const vector<int>& value,
    ll capacity
)
{
    assert(weight.size() == value.size());
    int total_value = accumulate(value.begin(), value.end(), 0);

```

```

const ll INF = numeric_limits<ll>::max() / 4;

vector<ll> dp(total_value + 1, INF);
dp[0] = 0; // 价值为 0 时不需要任何重量。

for (int i = 0; i < static_cast<int>(weight.size()); ++i)
{
    // 仍是 0/1 背包，所以价值倒序枚举。
    for (int current = total_value; current >= value[i]; --current)
    {
        if (dp[current - value[i]] == INF)
        {
            continue;
        }
        dp[current] = min(
            dp[current],
            dp[current - value[i]] + weight[i]
        );
    }
}

for (int answer = total_value; answer >= 0; --answer)
{
    if (dp[answer] <= capacity)
    {
        return answer;
    }
}
return 0;
}

```

8.4 最长上升子序列 (LIS)

`tails[len - 1]` 表示长度为 `len` 的严格上升子序列能够取得的最小末尾值。该数组不是原序列中的实际 LIS，但其长度就是答案。

```

template <class T>
int lis_length(const vector<T>& values)
{
    vector<T> tails;

    for (const T& value : values)
    {
        // lower_bound 对应“严格上升”；非严格上升改用 upper_bound。
        auto it = lower_bound(tails.begin(), tails.end(), value);
        if (it == tails.end())
        {
            tails.push_back(value);
        }
        else
        {
            // 用更小的末尾值替换，可为后续元素留下更大空间。

```

```

        *it = value;
    }
}
return static_cast<int>(tails.size());
}

```

8.5 最长公共子序列（LCS）与方案恢复

`dp[i][j]` 表示 `left` 的前 `i` 个字符和 `right` 的前 `j` 个字符的 LCS 长度。计算完成后沿状态来源倒推，可恢复一组具体方案。

```

string longest_common_subsequence(string_view left, string_view right)
{
    int n = static_cast<int>(left.size());
    int m = static_cast<int>(right.size());
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; ++i)
    {
        for (int j = 1; j <= m; ++j)
        {
            if (left[i - 1] == right[j - 1])
            {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            }
            else
            {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    string answer;
    int i = n;
    int j = m;
    while (i > 0 && j > 0)
    {
        if (left[i - 1] == right[j - 1])
        {
            // 这两个相等字符来自 dp[i - 1][j - 1]。
            answer.push_back(left[i - 1]);
            --i;
            --j;
        }
        else if (dp[i - 1][j] >= dp[i][j - 1])
        {
            --i;
        }
        else
        {
            --j;
        }
    }
}

```

```

    }

    reverse(answer.begin(), answer.end());
    return answer;
}

```

8.6 区间 DP：合并石子

`dp[left][right]` 表示把区间 `[left, right]` 合并成一堆的最小代价。最后一次合并一定会在某个 `middle` 处分成左右两段。

```

using ll = long long;

ll minimum_merge_cost(const vector<ll>& stones)
{
    int n = static_cast<int>(stones.size());
    if (n == 0)
    {
        return 0;
    }

    vector<ll> prefix(n + 1, 0);
    for (int i = 0; i < n; ++i)
    {
        prefix[i + 1] = prefix[i] + stones[i];
    }

    const ll INF = numeric_limits<ll>::max() / 4;
    vector<vector<ll>> dp(n, vector<ll>(n, 0));

    // 短区间必须先于长区间计算。
    for (int length = 2; length <= n; ++length)
    {
        for (int left = 0; left + length <= n; ++left)
        {
            int right = left + length - 1;
            ll interval_sum = prefix[right + 1] - prefix[left];
            dp[left][right] = INF;

            for (int middle = left; middle < right; ++middle)
            {
                dp[left][right] = min(
                    dp[left][right],
                    dp[left][middle]
                        + dp[middle + 1][right]
                        + interval_sum
                );
            }
        }
    }

    return dp[0][n - 1];
}

```

8.7 树形 DP：最大权独立集

选择若干树节点，要求任意两个被选节点不相邻，并最大化权值和。

- `dp[u][0]`：不选择 `u` 时，`u` 子树的最大答案；
- `dp[u][1]`：选择 `u` 时，`u` 子树的最大答案。

```
using ll = long long;

void tree_dp_dfs(
    int u,
    int parent,
    const vector<vector<int>>& tree,
    const vector<ll>& weight,
    vector<array<ll, 2>>& dp
)
{
    dp[u][0] = 0;
    dp[u][1] = weight[u];

    for (int v : tree[u])
    {
        if (v == parent)
        {
            continue;
        }

        tree_dp_dfs(v, u, tree, weight, dp);
        // 不选 u 时，孩子可以自由选择选或不选。
        dp[u][0] += max(dp[v][0], dp[v][1]);
        // 选择 u 后，与 u 相邻的孩子都不能选择。
        dp[u][1] += dp[v][0];
    }
}

ll maximum_weight_independent_set(
    const vector<vector<int>>& tree,
    const vector<ll>& weight,
    int root = 1
)
{
    // array 的值初始化会把两个状态都置为 0。
    vector<array<ll, 2>> dp(tree.size());
    tree_dp_dfs(root, 0, tree, weight, dp);
    return max(dp[root][0], dp[root][1]);
}
```

8.8 状态压缩 DP：最短哈密顿路径

从 0 号点出发，恰好访问每个点一次，并在 $n - 1$ 号点结束。 $dp[mask][last]$ 表示已访问集合为 $mask$ 、当前位于 $last$ 的最短距离。

```
using ll = long long;

ll shortest_hamilton_path(const vector<vector<ll>>& weight)
{
    int n = static_cast<int>(weight.size());
    if (n == 0)
    {
        return 0;
    }
    assert(n <= 20); // 状态数为  $2^n$ ,  $n$  再大通常无法承受。

    const ll INF = numeric_limits<ll>::max() / 4;
    int state_count = 1 << n;
    vector<vector<ll>> dp(state_count, vector<ll>(n, INF));
    dp[1][0] = 0; // mask == 1 表示只访问了 0 号点。

    for (int mask = 0; mask < state_count; ++mask)
    {
        for (int last = 0; last < n; ++last)
        {
            if (((mask >> last) & 1) == 0 || dp[mask][last] == INF)
            {
                continue;
            }

            for (int next = 0; next < n; ++next)
            {
                if ((mask >> next) & 1)
                {
                    continue;
                }
                if (weight[last][next] == INF)
                {
                    // INF 表示两点间没有可用边。
                    continue;
                }

                int next_mask = mask | (1 << next);
                dp[next_mask][next] = min(
                    dp[next_mask][next],
                    dp[mask][last] + weight[last][next]
                );
            }
        }
    }
    return dp[state_count - 1][n - 1];
}
```

8.9 数位 DP：统计指定数位和

下面统计 `[0, upper]` 中十进制数位和等于 `target_sum` 的整数个数。`tight` 表示当前前缀是否仍与上界前缀完全相同；只有 `tight == false` 的状态可以安全记忆化。

```
using ll = long long;

ll count_with_digit_sum(ll upper, int target_sum)
{
    // 负上界表示区间为空，直接返回 0。
    if (upper < 0 || target_sum < 0)
    {
        return 0;
    }

    string digits = to_string(upper);
    int length = static_cast<int>(digits.size());
    vector<vector<ll>> memo(
        length,
        vector<ll>(target_sum + 1, -1)
    );

    // position: 当前处理到第几位; sum: 已选数位和; tight: 是否贴住上界。
    function<ll(int, int, bool)> dfs = [&](int position, int sum, bool tight)
    {
        if (sum > target_sum)
        {
            return 0LL;
        }
        if (position == length)
        {
            return sum == target_sum ? 1LL : 0LL;
        }

        if (!tight && memo[position][sum] != -1)
        {
            return memo[position][sum];
        }

        int upper_digit = tight ? digits[position] - '0' : 9;
        ll ways = 0;
        for (int digit = 0; digit <= upper_digit; ++digit)
        {
            // 前导零自然参与递归，但不会改变数位和。
            ways += dfs(
                position + 1,
                sum + digit,
                tight && digit == upper_digit
            );
        }

        if (!tight)
    }
```

```

        {
            memo[position][sum] = ways;
        }
        return ways;
    };

    return dfs(0, 0, true);
}

11 count_with_digit_sum(11 left, 11 right, int target_sum)
{
    return count_with_digit_sum(right, target_sum)
        - count_with_digit_sum(left - 1, target_sum);
}

```

8.10 DAG 上的 DP：最长路径

拓扑序保证处理 u 时，所有指向 u 的前驱都已完成转移。下面返回以任意入度为 0 的点为起点、到每个点的最长边数；图中有环时返回 `nullptr`。

```

optional<vector<int>> longest_path_in_dag(
    const vector<vector<int>>& graph
)
{
    int n = static_cast<int>(graph.size()) - 1;
    vector<int> indegree(n + 1, 0);
    // 先统计入度，为 Kahn 拓扑排序做准备。
    for (int u = 1; u <= n; ++u)
    {
        for (int v : graph[u])
        {
            ++indegree[v];
        }
    }

    queue<int> q;
    vector<int> dp(n + 1, 0);
    for (int u = 1; u <= n; ++u)
    {
        if (indegree[u] == 0)
        {
            // 所有入度为 0 的点都可以作为一条路径的起点。
            q.push(u);
        }
    }

    int processed = 0;
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        ++processed;
    }
}

```

```

    for (int v : graph[u])
    {
        // 尝试用“到 u 的最长路径 + 边 u -> v”更新 v。
        dp[v] = max(dp[v], dp[u] + 1);
        if (--indegree[v] == 0)
        {
            q.push(v);
        }
    }
}

if (processed != n)
{
    // 还有点未进入拓扑序，说明图中存在有向环。
    return nullopt;
}
return dp;
}

```

9. 基础算法补充

本章整理 D:\ACM\模板笔记\板子.md 中适合赛时复用、且尚未被前文覆盖的基础模板。选择排序、冒泡排序等教学实现不再重复收录，比赛中直接使用 `std::sort`。

9.1 整数二分答案

`first_true` 查找第一个满足单调谓词的位置；要求区间右端点一定满足条件。

```

using ll = long long;

template <class Predicate>
ll first_true(ll left, ll right, Predicate check)
{
    assert(left <= right && check(right));

    while (left < right)
    {
        // 取下中位数；check(mid) 为真时保留 mid。
        ll mid = left + (right - left) / 2;
        if (check(mid))
        {
            right = mid;
        }
        else
        {
            left = mid + 1;
        }
    }
    return left;
}

```

```

template <class Predicate>
ll last_true(ll left, ll right, Predicate check)
{
    assert(left <= right && check(left));

    while (left < right)
    {
        // 取上中位数: check(mid) 为真时保留 mid。
        ll mid = left + (right - left + 1) / 2;
        if (check(mid))
        {
            left = mid;
        }
        else
        {
            right = mid - 1;
        }
    }
    return left;
}

```

9.2 双指针与滑动窗口

当数组元素均非负时，窗口右端点右移只会让区间和增大，因此左端点也只需单调右移。下面求和不超过 `limit` 的最长连续子数组。

```

using ll = long long;

int longest_window_sum_at_most(const vector<ll>& values, ll limit)
{
    assert(limit >= 0);

    int left = 0;
    int answer = 0;
    ll window_sum = 0;

    for (int right = 0; right < static_cast<int>(values.size()); ++right)
    {
        assert(values[right] >= 0);
        window_sum += values[right];

        while (left <= right && window_sum > limit)
        {
            // 当前窗口过大，移除最左端元素。
            window_sum -= values[left];
            ++left;
        }

        answer = max(answer, right - left + 1);
    }
    return answer;
}

```

9.3 归并排序统计逆序对

合并两个有序区间时，若右侧元素小于左侧当前元素，那么左侧剩余元素都与它构成逆序对。

```
using ll = long long;

template <class T>
ll inversion_count_impl(
    vector<T>& values,
    vector<T>& buffer,
    int left,
    int right
)
{
    if (right - left <= 1)
    {
        return 0;
    }

    int middle = (left + right) / 2;
    ll inversions = inversion_count_impl(values, buffer, left, middle)
        + inversion_count_impl(values, buffer, middle, right);

    int i = left;
    int j = middle;
    int k = left;
    while (i < middle || j < right)
    {
        if (j == right || (i < middle && values[i] <= values[j]))
        {
            buffer[k++] = values[i++];
        }
        else
        {
            buffer[k++] = values[j++];
            // values[i..middle) 全都大于刚取出的右侧元素。
            inversions += middle - i;
        }
    }

    copy(buffer.begin() + left, buffer.begin() + right, values.begin() + left);
    return inversions;
}

template <class T>
ll inversion_count(vector<T> values)
{
    // 按值传参，排序过程不会改变调用者的原数组。
    vector<T> buffer(values.size());
    return inversion_count_impl(
        values,
        buffer,
        0,
```

```

        static_cast<int>(values.size())
    );
}

```

9.4 回溯枚举排列

`path` 保存当前已经选择的数字，`used[x]` 表示数字 `x` 是否已经出现在当前排列中。总复杂度为 $O(n * n!)$ 。

```

template <class Callback>
void enumerate_permutations(int n, Callback process)
{
    vector<int> path;
    vector<char> used(n + 1, false);

    function<void()> dfs = [&]()
    {
        if (static_cast<int>(path.size()) == n)
        {
            // 找到一个完整排列，交给调用者处理。
            process(path);
            return;
        }

        for (int value = 1; value <= n; ++value)
        {
            if (used[value])
            {
                continue;
            }

            // 选择 value。
            used[value] = true;
            path.push_back(value);
            dfs();

            // 撤销选择，恢复进入本层递归前的状态。
            path.pop_back();
            used[value] = false;
        }
    };

    dfs();
}

```

9.5 回溯寻找子集和

下面返回一组元素下标，使所选元素之和等于 `target`。该模板不要求元素为正数，因此不使用“当前和超过目标就剪枝”的错误假设。

```

using ll = long long;

```

```

optional<vector<int>> find_subset_sum(
    const vector<ll>& values,
    ll target
)
{
    vector<int> chosen_indices;

    function<bool(int, ll)> dfs = [&](int index, ll remaining)
    {
        if (remaining == 0)
        {
            return true;
        }
        if (index == static_cast<int>(values.size()))
        {
            return false;
        }

        // 分支一：选择当前元素。
        chosen_indices.push_back(index);
        if (dfs(index + 1, remaining - values[index]))
        {
            return true;
        }
        chosen_indices.pop_back();

        // 分支二：不选择当前元素。
        return dfs(index + 1, remaining);
    };

    if (!dfs(0, target))
    {
        return nullopt;
    }
    return chosen_indices;
}

```

9.6 区间调度贪心

对半开区间 `[left, right)` 按右端点升序排序，每次选择最早结束且与已选区间不重叠的区间，可得到最多区间数量。

```

using ll = long long;

struct Interval
{
    ll left;
    ll right;
};

int maximum_non_overlapping_intervals(vector<Interval> intervals)
{

```

```

sort(intervals.begin(), intervals.end(), [](const Interval& lhs, const Interval& rhs)
{
    if (lhs.right != rhs.right)
    {
        // 最早结束的区间优先，为后续区间留下最多空间。
        return lhs.right < rhs.right;
    }
    return lhs.left < rhs.left;
});

ll current_end = numeric_limits<ll>::min();
int answer = 0;
for (const Interval& interval : intervals)
{
    assert(interval.left <= interval.right);
    if (interval.left >= current_end)
    {
        ++answer;
        current_end = interval.right;
    }
}
return answer;
}

```

复杂度速查

模板	时间复杂度	空间复杂度	关键前置条件
Miller-Rabin	$O(k \log n)$ 次模乘	$O(1)$	使用 64 位确定性底数集
Pollard-Rho	期望约 $O(n^{1/4})$	与递归深度相关	随机算法，输入 $n \geq 1$
线性筛	$O(n)$	$O(n)$	预处理上界已知
卡特兰数递推	$O(n)$	$O(n)$	精确值增长很快
循环引理找合法起点	$O(n)$	$O(1)$	序列总和大于 0
莫比乌斯线性筛	$O(n)$	$O(n)$	预处理上界已知
莫比乌斯反演	$O(n \log n)$	$O(n)$	已知所有因数和 $F(n)$
树状数组	单次 $O(\log n)$	$O(n)$	1-based
懒标记线段树	单次 $O(\log n)$	$O(n)$	1-based，区间赋值/求和
并查集	均摊 $O(\alpha(n))$	$O(n)$	1-based
异或线性基	单次 $O(64)$	$O(64)$	<code>uint64_t</code>
DFS / BFS	$O(n + m)$	$O(n)$	邻接表
Dijkstra	$O((n + m) \log n)$	$O(n + m)$	边权非负

模板	时间复杂度	空间复杂度	关键前置条件
SPFA	最坏 $O(nm)$	$O(n + m)$	可处理负边
Prim	$O((n + m) \log n)$	$O(n + m)$	无向图
Floyd	$O(n^3)$	$O(n^2)$	点数较少
二分图匈牙利算法	$O(VE)$	$O(V + E)$	已知左右部点集
KMP	$O(n + m)$	$O(m)$	0-based
Trie	单次 $O(s)$	与节点数成正比	当前版本仅含 a-z
AC 自动机	构建 $O(26S)$ ，查询 $O(T + S)$	$O(26S)$	当前版本仅含 a-z
0/1 / 完全背包	$O(nv)$	$O(v)$	容量 v 可承受
LIS	$O(n \log n)$	$O(n)$	严格上升用 <code>lower_bound</code>
LCS	$O(nm)$	$O(nm)$	当前版本需要恢复方案
区间 DP	$O(n^3)$	$O(n^2)$	合并石子模型
树形 DP	$O(n)$	$O(n)$	输入为树
状态压缩 DP	$O(2^n n^2)$	$O(2^n n)$	通常 $n \leq 20$
数位 DP	状态数乘每位 10 种转移	与状态数成正比	按数位从高到低搜索
DAG DP	$O(n + m)$	$O(n + m)$	图必须无环
整数二分答案	$O(\log R)$ 次检查	$O(1)$	<code>check</code> 具有单调性
双指针滑动窗口	$O(n)$	$O(1)$	当前模板要求元素非负
归并统计逆序对	$O(n \log n)$	$O(n)$	元素支持比较
排列回溯	$O(n * n!)$	$O(n)$	仅适合较小 n
子集和回溯	$O(2^n)$	$O(n)$	仅适合较小 n
区间调度	$O(n \log n)$	$O(n)$	半开区间，按右端点贪心